CHALLENGE DATA ENGINEER ARTEFACT

Comprendre simplement les Levels 1 et 2

De la generation de donnees a une plateforme analytique orchestree

But de ce document : comprendre ce qui est demande, pourquoi chaque composant existe, comment les donnees circulent et comment reconnaitre une solution terminee. Ce document n'est pas un tutoriel de code.

Perimetre : Level 1 - fondation batch, puis Level 2 - orchestration et architecture Medallion.

Principe de travail : comprendre d'abord, construire ensuite, puis savoir expliquer chaque choix.

1. Le challenge en une phrase

> **Construire une petite plateforme qui simule l'activite financiere de huit pays, stocke les donnees, les controle, les transforme en informations utiles et permet de les interroger en SQL.**

L'entreprise fictive WABA regroupe banque, assurance, mobile money et microfinance. Les donnees proviennent normalement de nombreux systemes. Dans le challenge, nous devons simuler ces sources, puis construire la plateforme qui les centralise.

La progression generale

Niveau	Question a laquelle il repond
Level 1	Peut-on generer, stocker, charger et interroger des donnees brutes fiables ?
Level 2	Peut-on automatiser les traitements et produire des indicateurs metier fiables ?
Level 3	Peut-on aussi traiter les evenements presque en temps reel ?
Level 4	Peut-on deployer, securiser, gouverner et superviser la plateforme ?

Notre objectif actuel

- Terminer un Level 1 propre et demonstrable.
- Ajouter ensuite le Level 2 sans reconstruire tout le Level 1.
- Garder Level 3 et Level 4 en tete uniquement pour ne pas creer de blocage futur.

2. Level 1 - ce qu'on vous demande reellement

Le Level 1 construit la chaine batch minimale de bout en bout.

Streamlit -> fichiers CSV -> MinIO raw-landing -> Spark -> tables Iceberg raw.* -> Trino SQL

Chaque bloc a une responsabilite unique

Composant	Role tres simple	Ce qu'il ne fait pas
Streamlit	Interface permettant de demander la generation de donnees fictives.	Il ne remplace pas Spark et ne fait pas l'analyse.
MinIO	Stockage objet compatible S3 pour recevoir les fichiers et stocker Iceberg.	Ce n'est pas une base SQL classique.
Spark	Lit, valide, transforme legerement et ecrit les donnees.	Il n'est pas l'interface utilisateur.
Iceberg	Format de tables de lakehouse avec metadonnees, partitions et transactions.	Ce n'est pas un moteur de calcul.
Trino	Moteur SQL qui rend les tables interrogeables.	Il ne genere pas les donnees.
Docker Compose	Lance tous les services de maniere reproductible.	Il ne remplace pas Kubernetes au Level 4.

Image mentale

Streamlit est le guichet. MinIO est l'entrepot de fichiers. Spark est l'atelier de controle et de chargement. Iceberg est l'organisation fiable des rayons. Trino est le comptoir SQL qui permet de consulter ce qui est range.

3. Etape 1.1 - l'application de generation

L'application Streamlit sert a simuler les systemes sources de WABA. L'utilisateur choisit quoi generer, pour quels pays, quelles entites, quel volume et quelle periode.

Selections attendues

- Famille de donnees : transactions bancaires, assurance, mobile money, remboursements de pret ou referentiels.
- Un ou plusieurs pays parmi CI, SN, ML, BF, GN, TG, BJ et GH.
- Une ou plusieurs lignes metier : BANK, INSURANCE, MOBILE_MONEY, MICROFINANCE.
- Nombre de lignes et intervalle de dates.
- Mode one-time : une generation immediate.
- Mode continu : une nouvelle generation toutes les 10 a 60 secondes.

Deux categories de donnees

Categorie	Exemples	Regle fondamentale
Referentiels	customers, accounts, branches, products	Ils doivent exister avant les transactions.
Evenements	bank transactions, insurance operations, mobile money, loan repayments	Chaque cle doit pointer vers un referentiel valide.

Coherence referentielle signifie simplement : ne jamais creer une transaction pour un client, un compte ou une agence qui n'existe pas.

Les donnees doivent aussi etre credibles : devise XOF dans les pays UEMOA, GHS au Ghana, statuts respectant les distributions indiquees, comptes de type LOAN pour les remboursements, et champs conditionnels coherents pour les sinistres.

4. Les fichiers et leurs dependances

Le nom du fichier transporte des informations utiles : type, pays, date et numero de lot. Un exemple est bank_txn_CI_20260101_01.csv.

Ordre logique

customers -> accounts -> transactions | branches -> transactions | products -> analyses metier

Relations essentielles

Flux	Cles a controler	Exemple de coherence
Banque	account_id, beneficiary_account, branch_id	Le compte debite et l'agence existent dans le meme contexte pays.
Assurance	customer_id, account_id	Le compte represente une police compatible avec le client.
Mobile money	sender_id, receiver_id	Les deux clients existent; leurs pays peuvent differer.
Pret	loan_account_id, customer_id	Le compte existe, appartient au client et son type vaut LOAN.

Securite

Les IBAN et numeros de compte ne doivent pas representer de vraies personnes. Les identifiants doivent etre fictifs ou pseudonymises. Aucun secret technique ne doit etre inscrit dans le code.

Reproductibilite

Une seed configurable rend les tests reproductibles : avec les memes parametres et la meme seed, on peut retrouver les memes distributions. Les identifiants de lots doivent toutefois eviter les collisions lors des executions reelles.

5. Etape 1.2 - MinIO

MinIO joue le role d'un stockage S3 local. Trois emplacements logiques sont demandes.

Bucket	Contenu	Moment
raw-landing	Fichiers CSV nouvellement produits.	Avant le traitement Spark.
lakehouse	Donnees et metadonnees des tables Iceberg.	Apres ecriture par Spark.
archive	Fichiers deja traites avec succes.	Uniquement apres validation et chargement reussis.

Un fichier ne doit jamais etre archive avant la fin reussie du traitement. En cas d'echec, il doit rester visible pour diagnostic ou etre place dans une zone de rejet documentee.

Organisation attendue

Le bucket raw-landing est organise par pays et type de donnees. Cela facilite la detection, le traitement selectif et les futurs backfills. Le chemin exact doit rester stable et documente.

Exemple conceptuel : raw-landing/country_code=CI/data_type=bank_transactions/date=2026-01-01/fichier.csv

6. Etape 1.3 - Spark et Iceberg

Spark prend chaque fichier disponible, lui applique un schema explicite, controle son contenu et ecrit les lignes valides dans Iceberg.

Un traitement minimal suit cet ordre

- 1. Identifier le fichier et son type.
- 2. Lire avec un schema declare, sans laisser Spark deviner les types.
- 3. Verifier les champs obligatoires, types, devises, pays et cles referentielles.
- 4. Separer les lignes valides des lignes rejetees et expliquer les raisons du rejet.
- 5. Eviter les doublons avec l'identifiant metier approprie.
- 6. Ecrire les lignes valides dans la table Iceberg cible.
- 7. Archiver le fichier uniquement si l'ecriture a reussi.

Les huit tables du Level 1

Referentiels	Evenements
raw.customers	raw.bank_transactions
raw.accounts	raw.insurance_operations
raw.branches	raw.mobile_money_payments
raw.products	raw.loan_repayments

Idempotence

Relancer exactement le meme fichier ne doit pas doubler les donnees. On controle les identifiants transaction_id, operation_id, payment_id ou repayment_id, et idealement aussi le fichier source.

Les tables sont partitionnees au minimum par country_code et date. La partition accelere les requetes filtrees, mais il ne faut pas la confondre avec une colonne supprimee : country_code et date restent des donnees metier visibles.

7. Etape 1.4 - Trino et la preuve que le Level 1 fonctionne

Trino doit se connecter au catalogue Iceberg et rendre les huit tables interrogeables en SQL.

Requetes de preuve

- Compter les lignes par table et verifier que les huit tables existent.
- Calculer les volumes et montants par pays.
- Compter par entity_type.
- Verifier qu'aucune transaction n'a une cle referentielle orpheline.
- Relancer un fichier, puis montrer que le nombre de lignes ne double pas.

Definition of Done - Level 1

Question	Reponse attendue
L'interface fonctionne-t-elle ?	Les modes one-time et continu produisent les quatre flux et les referentiels.
Les fichiers arrivent-ils dans MinIO ?	Oui, dans des chemins par pays et type.
Les relations sont-elles valides ?	Aucune cle transactionnelle orpheline.
Les huit tables existent-elles ?	Oui, peuplees et requetables dans Trino.
Le pipeline est-il idempotent ?	Une relance n'ajoute aucun doublon.
Le lancement est-il simple ?	Un docker compose up lance la stack documentee.
Un evaluateur peut-il reproduire ?	Le README suffit depuis un clone propre.

8. Level 2 - ce qui change

Le Level 1 prouve que la chaine fonctionne. Le Level 2 transforme cette chaine en pipeline automatise et ajoute des couches de qualite et d'analyse.

> **Airflow orchestre : ingestion -> Bronze -> Silver -> Gold -> reporting reglementaire**

Airflow ne traite pas lui-meme les gros volumes. Il decide quel job Spark demarrer, dans quel ordre, avec quels parametres, et quoi faire en cas d'echec.

Les quatre DAGs

DAG	Declenchement	Responsabilite
dag_ingest_raw	Nouveaux fichiers ou planification	Charger les donnees vers Bronze pour les pays selectionnes.
dag_bronze_to_silver	Apres ingestion reussie	Nettoyer, dedupliquer, enrichir et consolider.
dag_silver_to_gold	Apres Silver reussi	Calculer les sept tables de KPI.
dag_regulatory_report	Chaque jour a 00h30 UTC retenu	Produire les agregats BCEAO et CIMA a J+1.

Pourquoi 00h30 ?

Le texte mentionne aussi 01h00, mais le critere d'evaluation exige 00h30 UTC. La solution doit documenter cette ambiguite et suivre le critere evalue, sauf clarification d'Artefact.

9. Comprendre Bronze, Silver et Gold

Couche	Question simple	Contenu
Bronze	Qu'avons-nous recu ?	Donnees proches de la source, schema valide, avec traces d'ingestion.
Silver	Quelles donnees sont propres et comparables ?	Donnees dedupliquees, normalisees, enrichies et converties en EUR.
Gold	Quelles reponses metier voulons-nous servir ?	Agregats et KPI directement utilisables dans Trino et plus tard Superset.

Bronze

Bronze conserve la fidelite a la source tout en imposant un schema minimal. Il faut ajouter des champs techniques comme ingestion_timestamp et source_file pour tracer l'origine.

Silver

Silver applique les regles communes : suppression des doublons, traitement des valeurs nulles, jointure avec customers/accounts/branches/products, gestion des cas aberrants et conversion des montants en EUR. C'est la couche de confiance reutilisable.

Gold

Gold ne conserve pas tous les details. Elle organise les resultats selon les questions metier : par jour, mois, pays, produit, segment ou corridor. Ces tables doivent etre rapides et faciles a comprendre.

> **Raw et Bronze semblent se chevaucher dans l'enonce. Une interpretation defensable est : raw-landing contient les fichiers recus; Bronze est la premiere table Iceberg validee et tracable. Les tables raw.* du Level 1 peuvent etre migrees ou mappees clairement vers bronze.* au Level 2.**

10. Les sept tables Gold

Table	Ce qu'elle mesure	Grain principal
gold.daily_transaction_volume	Volume et montant par type; activite commerciale.	Jour, pays, entite, type
gold.npl_ratio_by_country	Part des encours de prets en defaut.	Pays, type de pret
gold.customer_arpu_monthly	Revenu moyen par client.	Mois, pays, segment
gold.loss_ratio_by_product	Sinistres payes divises par primes acquises.	Mois, pays, produit
gold.claims_processing_time	Delai moyen de traitement des sinistres.	Pays, ligne IARD/Vie
gold.mobile_money_daily_flow	Volume, montant, echecs et utilisateurs actifs.	Jour, pays, operateur
gold.cross_border_transfers	Flux entrants/sortants et corridors.	Jour ou mois, pays source/destination

Point crucial

Le generateur du Level 1 doit contenir tous les champs permettant ces calculs. Si les donnees sources ne permettent pas un KPI, le probleme ne peut pas etre repare uniquement dans Gold.

Exemples

- Le NPL exige amount_due, amount_paid et un statut DEFAULT coherent.
- L'ARPU exige les revenus par client : frais et interets doivent etre identifiables.
- Le loss ratio exige de distinguer les primes acquises et les sinistres payes.
- Les corridors exigent sender_country et receiver_country.

11. Qualite d'orchestration attendue

Un DAG n'est pas seulement une liste de commandes. Il doit rendre l'exploitation sure et explicable.

- Dependances explicites : une couche ne demarre que si la precedente a reussi.
- Retries limites et delais raisonnables pour les erreurs temporaires.
- Alerte ou journal clair lorsqu'un traitement echoue.
- Parametres par pays et par periode pour permettre un backfill selectif.
- Airflow Connections, Variables ou variables d'environnement pour les configurations.
- Aucun mot de passe ou secret dans le code, les DAGs ou le depot Git.

Backfill

> **Un backfill signifie rejouer une periode historique manquante ou corrigee. Exemple : recalculer uniquement le Senegal du 1er au 7 janvier, sans retraiter les huit pays.**

La reexecution doit rester idempotente dans toutes les couches. Cela implique des ecritures de type MERGE, remplacement de partitions controle ou strategie equivalente, et non un append aveugle.

12. Definition of Done - Level 2

Critere	Preuve simple a montrer
4 DAGs operationnels	Vue Airflow verte, dependances visibles, execution par pays.
Architecture Medallion	Schemas bronze.*, silver.* et gold.* distincts dans Iceberg/Trino.
Silver de qualite	Test de deduplication, conversion EUR, jointures et valeurs nulles.
7 KPI Gold	SELECT sur chaque table avec resultats plausibles et filtre par pays.
Secrets externalises	Depot sans credentials; configuration via environnement/Connections.
Reporting J+1	DAG programme a 00h30 UTC et agregats BCEAO/CIMA disponibles.

Resultats plausibles

- NPL genere entre environ 3 % et 8 % selon les pays.
- Loss ratio genere entre environ 50 % et 85 % selon les pays.
- Les sommes Silver et Gold sont explicables depuis les donnees sources.
- Les conversions en EUR utilisent une table ou regle de taux documentee et datee.

Attention

Une table qui retourne des lignes n'est pas necessairement correcte. Il faut pouvoir expliquer le grain, la formule, les filtres, le traitement des echecs, la devise et les cas nuls de chaque KPI.

13. Les erreurs de conception a eviter

- Commencer par l'interface Streamlit sans definir les schemas et dependances.
- Generer au hasard des cles et esperer qu'elles correspondent aux referentiels.
- Utiliser uniquement l'inference de schema Spark.
- Archiver un fichier avant d'avoir confirme l'ecriture Iceberg.
- Faire des append sans strategie d'idempotence.
- Melanger Bronze, Silver et Gold dans un seul script impossible a tester.
- Coder les secrets, chemins, pays, dates ou taux de change en dur.
- Produire des KPI dont le grain et la formule ne sont pas documentes.
- Tester seulement avec le volume complet : les iterations deviendraient trop lentes.
- Ajouter trop de technologies non demandees avant d'obtenir un chemin simple de bout en bout.

Deux profils de volume

Un profil dev tres petit permet d'iterer rapidement. Un profil nominal reproduit les volumes demandes pour la validation et la demonstration. Les deux utilisent exactement les memes schemas et regles.

14. Ordre de construction recommande

Etape	Resultat observable
1. Contrats de donnees	Schemas, enums, cles, regles conditionnelles et grains documentes.
2. Petit generateur sans UI	Quelques lignes coherentes pour chaque fichier.
3. MinIO	Buckets et chemins crees; upload et lecture testes.
4. Spark + Iceberg	Une table de bout en bout, puis generalisation aux huit tables.
5. Trino	Requetes SQL et controles de coherence.
6. Streamlit	Interface branchee sur une logique de generation deja testee.
7. Idempotence et erreurs	Relances, rejets et archivage prouves.
8. Airflow	DAG d'ingestion, puis enchainement Bronze/Silver/Gold.
9. KPI Gold	Sept tables testees avec resultats plausibles.
10. Documentation et demo	README reproductible, write-up et scenario video.

La meilleure demonstration n'est pas celle qui montre le plus d'ecrans. C'est celle qui suit une donnee depuis sa generation jusqu'au KPI et prouve les controles importants.

15. Questions a savoir expliquer en entretien

- Pourquoi MinIO plutot qu'un dossier local ?
- Pourquoi Iceberg au-dessus du stockage objet ?
- Quelle difference entre Spark et Trino ?
- Comment garantissez-vous qu'aucune transaction n'est orpheline ?
- Comment prouvez-vous l'idempotence ?
- Pourquoi separer Bronze, Silver et Gold ?
- Comment un backfill par pays fonctionne-t-il ?
- Comment traitez-vous une ligne invalide sans perdre sa trace ?
- D'ou vient le taux EUR et comment le rendez-vous reproductible ?
- Comment verifier qu'un KPI Gold est correct ?
- Que changeriez-vous pour passer du Docker Compose a Kubernetes ?

Reponse attendue

Pour chaque choix, etre capable de donner : le besoin, la decision, le fonctionnement, la preuve, la limite connue et l'evolution possible. Cette structure est plus convaincante qu'une liste de technologies.

16. Resume final

Level 1 construit un chemin batch fiable de la source simulee jusqu'au SQL. Level 2 automatise ce chemin et transforme les donnees en indicateurs metier et reglementaires.

Level 1 est reussi si

- les donnees sont coherentes et multi-pays ;
- les fichiers sont stockes et traites sans doublons ;
- les huit tables Iceberg sont accessibles dans Trino ;
- la stack est reproductible avec Docker Compose.

Level 2 est reussi si

- Airflow orchestre correctement les quatre DAGs ;
- Bronze, Silver et Gold ont des responsabilites distinctes ;
- les sept tables Gold donnent des resultats plausibles et explicables ;
- les traitements sont rejouables par pays et sans secrets codes en dur.

Votre objectif n'est pas seulement de faire fonctionner la plateforme. Vous devez etre capable de suivre une donnee, justifier chaque transformation et demontrer les garanties de qualite.